

RESUMEN P1 MEDINA

GIT

Es un sistema de control de versiones distribuido. Permite trabajar sin conexión y ofrece independencia antes de sincronizar con otros nodos. Cada usuario e integrante del equipo tiene una copia local completa de su propio repositorio con el historial de cambios, a diferencia de los sistemas de control de versiones centralizados, que existe un solo repositorio que almacena todo el código fuente y el historial de cambios de manera única en un servidor dedicado. Todos los desarrolladores sincronizan sus archivos desde este punto central (ejemplo: SVN, TFS).

Componentes locales

- Directorio de trabajo local.
- Índice: área de ensayo (stage, o caché).
- Historial: almacena versiones o commits.
- Head: marca la posición actual del proyecto.

Directorio -(add)-> Índice -(commit)-> Historial
(-----Máquina Local-----)

Ramas

- Git facilita la creación de ramas (master = rama inicial).
- Operaciones: crear ramas (branch), cambiar a ramas (checkout), combinar (merge), etiquetar (tag).
- Existen múltiples estilos de ramificación.

Operaciones de ramas

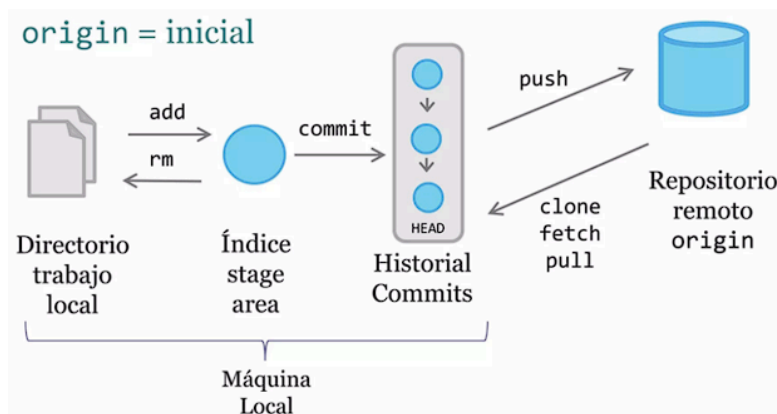
- **branch - gestión de ramas**
 - <r> = crea la rama <r>.
 - -d <r> = borra rama <r> (safe, no borra si hay merge pendientes).
 - -D <r> = borra rama <r> (unsafe, borra rama y sus commits).
 - -m <r> = renombra rama actual a <r>.

- **checkout - cambiar a una rama**
 - `<r>` = cambia a rama existente `<r>`.
 - `master` = cambia a rama master.
 - `-b <r>` = crea rama `<r>` y cambia a ella.
- **merge - combinar dos ramas**
 - `<r>` = combina rama actual con `<r>`.
 - `--no--ff <r>` = combina generando commit de mezcla (safe).

Tipos de merge

- Fast-forward: cuando hay camino lineal entre rama actual y rama a mezclar.
- 3 way merge: cuando las ramas divergen y no se puede hacer fast-forward (si aparecen conflictos: `git status`, se corrige, `git add .`, `git commit`).

Se pueden conectar repositorios remotos.



Funcionamiento básico

- **config:** `git config --global user.name <name>` (declara el nombre de usuario, también puede declarar email, alias, herramientas, etc).
- **init:** `git init` (transforma el directorio actual en repositorio git, se crea `.git`)
- **status:** `git status` (muestra ficheros staged -en índice pero no en historial-, unstaged -modificados pero no añadidos a índice- y untracked -en directorio de trabajo-).

- **add:** git add <fichero> / <dir> / --all (<fichero> -añade fichero o directorio al índice-, all -añade / borra ficheros-. El índice almacena copias de los ficheros antes de ser incluidos en el historial).
- **commit:** git commit -m "mensaje" (añade ficheros del índice al historial, crea una snapshot -SHA01-, puede recuperarse posteriormente).
- **log:** git log (muestra historial, --oneline -resumen en 1 línea-, --stat -estadísticas-, -p -camino completo con diff-, --author="expr" -commits de un autor-, --grep="expr" -busca commits-, --graph --decorate --oneline -grafo de cambios-).
- **diff:** git diff (directorio de trabajo vs índice, --cached -índice vs commit-, head -directorio de trabajo vs commit-).
- **clone:** git clone <repo> (clona el repositorio en máquina local, el repositorio puede estar en una máquina remota).

Deshaciendo cambios (cada comando actúa sobre una parte diferente)

- **checkout:** git checkout (cambia a directorio de trabajo, <c> -cambiar a commit-, <c> <f> -recupera fichero <f> de commit <c>-, se utiliza para cambiar a diferentes ramas).
- **revert:** git revert <c> (recupera commit <c>, operación segura, añade la versión recuperada al historial).
- **reset:** git reset (deshacer cambios, operación no segura, --hard -deshace cambios en índice y directorio de trabajo-, <c> -deshace cambios y recupera commit <c>-).
- **clean:** git clean (borra ficheros locales, -f -borra ficheros untracked-, -n -muestra que ficheros se borrarán-).

Repositorios remotos

- **remote - muestra repositorios remotos**
 - add <nombre> <uri> = conecta un nuevo repositorio.
 - rm <nombre> = elimina la conexión.
 - rename <nombre> <n_nombre> = renombra el remoto.
- **push - enviar**
 - <remoto> <rama> = envía commits locales al remoto.
 - <remoto> --all = envía todas las ramas.

- **fetch - traer (operación segura)**
 - <remote> = descarga todas las ramas.
 - <remote> <rama> = descarga la rama <rama>.
- **pull - traer y mezclar**
 - <remote> = trae repositorio remoto y lo mezcla.

Github es una herramienta de codificación social, permite alojar y colaborar en proyectos públicos o privados.

Dentro de los clientes GIT (Git GUI y Terminal) tenemos **GitHub Desktop**, **GitKraken** y **Fork**, entre otros.

Arquitectura Cliente - Servidor

El modelo cliente-servidor es un sistema distribuido que divide el trabajo en dos roles claros

- El cliente solicita un servicio o recurso.
- El servidor procesa la solicitud y devuelve una respuesta.

Separar los roles permite escalar cada parte por separado, reutilizar el servidor con múltiples clientes, y aislar fallos.

Protocolos de transferencia

- **HTTP (Cliente - Servidor basado en solicitudes sin estado)**
 - Datos viaja en texto plano.
 - Puerto por defecto: 80.
 - Vulnerable a interceptación.
 - Suficiente para entornos de pruebas.
- **HTTPS (HTTP + Capa de seguridad TLS / SSL)**
 - Datos cifrados.
 - Puerto por defecto: 443.
 - Verifica integridad del servidor (certificados).
 - Estándar en producción.

Estructura de petición HTTP/S

GET /api/usuarios // Ruta del recurso solicitado

Host: mipaginaweb.com // Dominio del servidor

Accept: application/json // Tipo de contenido esperado

Authorization: Bearer eyJhbGciOiJIUzI1Ni... // Token de autenticación

Estructura de respuesta HTTP/S

HTTP/ 200 OK // Código de estado

Content-Type: application/json // Tipo de contenido devuelto

```
{  
  "id": 1,  
  "nombre": "Juan Pérez",  
  "rol": "admin",  
  "email": "juanperez@mipaginaweb.com"  
}
```

Roles y responsabilidades de cada capa

- **Presentación:** Interfaz con el usuario (HTML, CSS, UX / UI).
- **Lógica de negocio:** Reglas, validaciones y procesos que resuelven el problema del dominio (controladores ASP.NET).
- **Datos:** Persistencia y acceso a la información (bases de datos, archivos, APIs externas, entity framework).

Cada capa se comunica solo con la adyacente, no se saltan.

Desacoplamiento: cada capa cambia sin afectar a las demás, siempre que se respete el contrato (interfaz) entre ellas.

Mantenimiento: errores y mejoras se localizan más rápido cuando cada componente tiene una responsabilidad única.

Escalabilidad: se puede reforzar solo la capa que lo necesita sin duplicar todo el sistema.

El frontend y backend son los dos componentes que constituyen el núcleo de cualquier aplicación web moderna.

Frontend: es lo que los usuarios ven e interactúan en una web (UI / UX, capacidad de respuesta, interactividad, compatibilidad entre navegadores).

Patrón común

- HTML
- CSS
- JavaScript

Backend: es la lógica interna, gestionando las tareas (lógica, gestión de base de datos, desarrollo de APIs, seguridad).

Patrón común

- ASP.NET Core 8.0 - Framework principal
- Entity Framework Core - ORM para base de datos
- SQL Server / PostgreSQL / MySQL - Bases de datos relacionales
- JWT Bearer - Autenticación basada en tokens
- BCrypt - Encriptación de contraseñas
- AutoMapper - Mapeo de objetos
- Swagger / OpenAPI - Documentación de API

ASP.NET Core es un framework web moderno, multiplataforma y de alto rendimiento, desarrollado por Microsoft para construir aplicaciones web, APIs y microservicios. Es la evolución de ASP.NET Framework tradicional.

Características y beneficios de ASP.NET Core

- Alto rendimiento
- Multiplataforma
- Modular y flexible
- Seguridad integrada
- Moderno
- Testeable
- Escalable

Estructura típica de un proyecto

MiProyecto/

- Controllers/ # Controladores de la API
 - Manejo de requests HTTP y respuestas
 - Validación de entrada
 - Coordinación con servicios
- Models/ # Modelos de datos y DATOs
 - Representación de datos
 - Validaciones
 - Relaciones entre entidades
- Services/ # Lógica de negocio
 - Orquestación de operaciones
 - Transformación de datos
- Repositories/ # Acceso a datos
 - Abstracción de la base de datos
 - Operaciones CRUD
- Config/ # Configuración de base de datos
- Utils/ # Utilidades y helpers
- Migrations/ # Migraciones de Entity Framework
- Program.cs # Punto de entrada de la aplicación
- appsettings.json # Configuración de la aplicación

Flujo de request en ASP.NET Core

- **Endpoints Comunes**
 - CRUD: Create, Read, Update, Delete
 - Authentication: Login, Register, Logout
 - Authorization: Endpoints protegidos por roles
 - Health Checks: Verificación del estado del sistema
 - File Upload / Download: Manejo de archivos
- **Códigos de estado HTTP**
 - 200 Ok - Operacion exitosa
 - 201 Created - Recurso creado exitosamente
 - 204 No Content - Recurso sin contenido
 - 301 Moved Permanently - Recurso redireccionado
 - 304 Not Modified - Recurso no redireccionado
 - 400 Bad Request - Datos inválidos
 - 401 Unauthorized - No autenticado

- 403 Forbidden - Sin permisos
- 404 Not Found - Recurso no encontrado
- 500 Internal Server Error - Error del servidor
- 503 Service Unavailable - Servicio no disponible

Servicios Web y API REST

Un servicio web expone la funcionalidad de una aplicación para que otra la consuma a través de la red, sin conocer su código interno. El cliente inicia la comunicación, el servidor la atiende y responde. El protocolo que hace posible esta comunicación en la web es HTTP.

Una API REST es un conjunto de reglas (o estilo arquitectónico) para usar HTTP de forma consistente al exponer recursos. Todo lo que sigue (verbos, códigos de estado, JSON) vive dentro del protocolo HTTP.

SOAP vs REST	SOAP	REST
Formato de datos	Solo XML	JSON, XML, texto
Transporte	Requiere HTTP/SMTP	Usa HTTP Directamente
Contrato	WSDL obligatorio	Sin contrato rígido
Complejidad	Alta, muy estructurado	Liviano y flexible
Uso típico hoy	Sistemas legales / bancos	Apps web y móviles

Principios de REST

- **Cliente - Servidor:** Responsabilidades separadas, la interfaz y el almacenamiento de datos evolucionan de forma independiente.
- **Sin estado (Stateless):** Cada petición contiene toda la información necesaria. El servidor no guarda sesión del cliente.
- **Recursos identificables:** Todo se modela como un recurso, accesible mediante una URI única (ejemplo: /api/productos/5).
- **Interfaz uniforme:** Se interactúa con los recursos siempre de la misma manera, usando los verbos HTTP estándar.

- **Cacheable:** Las respuestas pueden marcarse como cacheable para mejorar rendimiento.

Un recurso es “algo” con lo que se puede interactuar: un producto, un usuario, un pedido. El mismo endpoint cambia de comportamiento según el verbo HTTP usado.

Verbos HTTP

- **GET:** Obtener uno o varios recursos. No modifica datos.
- **POST:** Crear un nuevo recurso.
- **PUT:** Actualizar (reemplazar) un recurso existente por completo.
- **PATCH:** Actualizar parcialmente un recurso existente.
- **DELETE:** Eliminar un recurso.

Ciclo de vida de petición REST

- El cliente arma la petición (define método, URI, headers y body).
- Viaja por la red (la petición llega al servidor vía HTTP / HTTPS).
- El servidor la enruta (El framework decide qué controlador la atiende).
- Se procesa la lógica (Se valida, se consulta con la base de datos y se aplican reglas de negocio)
- Se devuelve la respuesta (Código de estado + body vuelven al cliente).

JSON (JavaScript Object Notation) es el formato principal que usan las páginas web y aplicaciones móviles para comunicarse con el servidor. Es liviano, fácil de leer y de generar. Se arma con pares clave-valor, arrays, y objetos anidados.

Buenas prácticas al diseñar una API REST

- Nombrar los recursos con sustantivos en plural (/clientes, /pedidos).
- Usar el verbo HTTP correcto para cada acción.
- Devolver el código de estado que representa el resultado.
- Versionar la API (ej: /api/v1/...) para no romper clientes existentes.
- Usar HTTPS y autenticación (tokens) para proteger los endpoints.
- Pagar las colecciones grandes en lugar de devolver todo de una vez.

Entity Framework Core

DbContext -> DbSet<T> -> Base de datos

Un ORM traduce mundos

- Clases <-> Tablas
- Atributos <-> Columnas
- Instancias <-> Filas
- Referencia <-> Clave foránea

Cada instancia de un objeto es una fila en la base de datos.

Resuelve el “impedance mismatch” entre objetos y datos relacionales.

Active Record vs Data Mapper

- EF Core sigue el patrón Data Mapper: el DbContext es la sesión, y las entidades no saben nada de persistencia.

Code First y Database First

- El modelo puede partir del código o de una base existente.

Arquitectura de EF Core

- **Tu aplicación:** código C# que trabaja con objetos normales (POCOs).
- **DbContext:** representa la sesión, agrupa los DbSet y coordina los cambios. **Es la pieza central**, en él se registran los DbSet<T> (uno por entidad) y se llama a SaveChanges() para confirmar cualquier cambio pendiente contra la base.
- **Change Tracker:** detecta qué objetos cambiaron desde que se cargaron.
- **Proveedor:** traduce LINQ al dialecto SQL del motor elegido.
- **Base de datos:** SQL Server, PostgreSQL, SQLite, etc.

```
public class Autor  
{
```

```

public int Id { get; set; }
public string Nombre { get; set; }

public List<Libro> Libros
    { get; set; }
}

public class Libro
{
    public int Id { get; set; }
    public string Titulo { get; set; }

    public int AutorId { get; set; }
    public Autor Autor { get; set; }
}

```

Modelante entidades y relaciones

- **Propiedades de navegación:** Libros y autor permite navegar la relación desde cualquiera de los dos lados (en código C# normal).
- **Clave foránea explícita:** AutorId es la clave foránea. Al llamarse <Nombre de la navegación> + Id, EF Core la detecta sola.
- **Relación uno a muchos:** Un Autor tiene muchos Libros; cada Libro pertenece a un solo Autor. Es la relación más común, aunque también existen otras como uno a uno con una FK única y muchos a muchos con una entidad de unión.

CRUD con EF Core

Create	Read	Update	Delete
<pre> var libro = new Libro { Título = "Rayuela", AutorId = 3 }; context.Libros.Ad </pre>	<pre> var libro = context.Libros .FirstOrDefault(l => l.Id == 5); var todos = context.Libros .ToList(); </pre>	<pre> var libro = context.Libros .Find(5); libro.Título = "Nuevo título"; context.SaveChan ges(); </pre>	<pre> var libro = context.Libros .Find(5); context.Libros.Re move(libro); context.SaveChan </pre>

d(libro); context.SaveChanges();			ges();
-------------------------------------	--	--	--------

Convenciones de EF Core (cuánto puede inferir el modelo sin decir nada)

- **Clave primaria:** propiedad llamada id o <NombreClase>.
- **Nombre de tabla:** nombre del DbSet<T> en el DbContext.
- **Tipo de columna:** hereda de C#: int -> int, string -> nvarchar(max).
- **Clave foránea:** <NombreNavegacion> + Id.
- **Campos requeridos:** los no nulos de C# se mapean como NOT NULL.
- **Relaciones:** se detectan automáticamente a partir de las propiedades de navegación.

Si la convención no es suficiente para expresar una regla de negocio, entran Data Annotations o Fluent API.

Data Annotations se basa en atributos sobre la propia clase, mientras que Fluent API tiene una configuración centralizada en el DbContext

Data Annotations	Fluent API
<pre>public class Libro { [Key] public int Id { get; set; } [Required, MaxLength(200)] public string Titulo { get; set; } }</pre>	<pre>modelBuilder.Entity<Libro>(e => { e.Property(l => l.Titulo) .IsRequired() .HasMaxLength(200); });</pre>
Rápido y legible para reglas simples, pero mezcla configuración de persistencia con la clase de dominio.	Más verboso, pero con mucho más poder: relaciones complejas, índices, restricciones, con entidades limpias.

Fluent API en profundidad

- **ToTable{...}** = Fija el nombre de la tabla en la base de datos.
- **HasKey{...}** = Define (o redefine) la clave primaria.

- **HasOne / WithMany** = Configura relaciones explícitamente en ambos sentidos.
- **HasForeignKey(...)** = Indica que propiedad actúa como clave foránea.
- **HasIndex(...).IsUnique()** = Crea índices y restricciones de unicidad.

Code First (clases C# a esquema de base de datos versionado)

- Definir las entidades (clases POCO y si hace falta configurar con Fluent API).
- Generar la migración (dotnet ef migrations add Nombre para crear archivo con los cambios).
- Revisar código generado (métodos Up() y Down()) para aplicar y revertir cambios).
- Aplicar a la base (dotnet ef database update para ejecutar las migraciones pendientes).

Database First (de esquema ya existente a clases del modelo)

- Partir de la base existente (ya hay tablas, relaciones y datos).
- Instalar herramientas (paquetes .Design y del proveedor correspondiente).
- Correr el scaffolding (el comando inspecciona el esquema y genera clases + DbContext).
- Ajustar lo generado (renombrar, mover archivos o completar con Fluent API).

¿Cuál elegir?

Factor	Code First	Database First
Proyecto nuevo (greenfield)	✓	—
Base de datos existente / legacy	—	✓
Equipo de desarrollo controla el esquema	✓	—
DBA administra esquema por separado	—	✓
Esquema versionado junto a código	✓	—

Arrancar rápido sobre base ya armada	—	✓
--------------------------------------	---	---



Un **DTO** (Data Transfer Object) es un objeto simple, sin lógica de negocio, que se usa para transportar datos entre capas. No es una entidad EF Core, es una copia exacta de lo que el cliente necesita ver o enviar.

Entidad (EF Core) - uso interno	DTO - contrato público de la API
<pre>public class Libro { public int Id { get; set; } public string Titulo { get; set; } public decimal CostoInterno { get; set; } public int AutorId { get; set; } public Autor Autor { get; set; } }</pre>	<pre>public class LibroDto { public int Id { get; set; } public string Titulo { get; set; } public string NombreAutor { get; set; } } // Sin CostoInterno, sin ciclos</pre>

- Oculta datos sensibles (columnas que el cliente no debería ver).
- Evita referencias circulares (Serializar Libro -> Autor -> Libros -> Autor -> Autor... rompe la serialización JSON).
- Desacopla el contrato de la API (cambiar el esquema interno no debería romper a los clientes que la consumen).
- Una forma por caso de uso (un DTO para crear -sin id-, otro para listar -resumido-, otro para el detalle).

AutoMapper es una librería de mapeo objeto-objeto para .NET - se usa junto a EF Core para convertir automáticamente entidades del modelo en los DTO anteriores, evitando escribir a mano el mismo código de asignación de propiedades una y otra vez.

class Libro (Entity) -> AutoMapper -> LibroDto

- Menos código repetitivo (en vez de asignar cada propiedad a mano, una sola línea mapea el objeto completo).

- Entidades y DTOs desacoplados (el esquema interno puede cambiar sin romper el contrato público de la API).
- Convención + configuración (mapea automáticamente propiedades con el mismo nombre, y permite reglas a medida para el resto).
- Se integra con LINQ (ProjectTo<T>()) traduce el mapeo a la consulta SQL, trayendo solo las columnas necesarias).

AutoMapper en la práctica

Definir el mapeo (Profile)	Usarlo con inyección de dependencias
<pre> public class MappingProfile : Profile { public MappingProfile() { CreateMap<Libro, LibroDto>(); CreateMap<Autor, AutorDto>() .ForMember(d => d.CantidadLibros, opt => opt.MapFrom(src => src.Libros.Count)); } } // Program.cs builder.Services.AddAutoMapper(typeof(MappingProfile)); </pre>	<pre> public class LibrosController { private readonly AppDbContext _ctx; private readonly IMapper _mapper; public async Task<List<LibroDto>> GetLibros() { return await _ctx.Libros .ProjectTo<LibroDto>(_mapper.ConfigurationProvider) .ToListAsync(); } } // ProjectTo, a diferencia de Map, arma // el SELECT solo con las columnas que el // DTO necesita. </pre>

Buenas prácticas con EF Core

- No exponer entidades en tu API (usar DTOs o records).
- AsNoTracking() en solo lectura (evitar overhead de change tracker).
- Include / ThenInclude (cargar relaciones explícitas para evitar N+1).
- Migraciones chicas y revisadas (Up() / Down() generado).
- Evita EnsureCreated() en producción (solo para pruebas rápidas).
- Transacciones explícitas (varias operaciones juntas -> manualmente).

En resumen

- El DbContext es el centro de todo: agrupa los DbSet<T> y confirma cambios con SaveChanges().
- Las convenciones resuelven la mayoría de casos. Configurar explícitamente solo cuando haga falta.
- Data Annotations es rápido para reglas simples; Fluent API da control total y mantiene las entidades limpias.
- Code First versiona el esquema junto al código con migraciones; Database First parte de una base existente con scaffolding.
- La elección entre ambos depende de quien controla el esquema: el equipo de desarrollo o un DBA.

ORM (Object-Relational Mapping)

class Usuario <-> TABLE usuarios

Un **ORM** es una técnica (y librería) que convierte automáticamente datos entre un sistema orientado a objetos y una base de datos relacional, permitiendo trabajar con objetos en lugar de escribir SQL a mano.

Impedance mismatch -> dos formas distintas de organizar datos

Mundo de los objetos	Mundo relacional
Herencia entre clases	Tablas planas de filas y columnas
Referencias y punteros entre objetos	Claves primarias y foráneas
Colecciones anidadas (listas, sets)	Relaciones expresadas con JOINS
Identidad de objeto (misma instancia)	Sin herencia nativa de estructuras
Encapsulamiento de comportamiento	Datos separados del comportamiento

Ventajas de usar ORM

- Productividad (menos SQL repetitivo; el CRUD básico se escribe en pocas líneas de código orientado a objetos.)

- Portabilidad (el mismo código puede funcionar en distintos motores de base de datos con mínimos cambios)
- Seguridad (genera consultas parametrizadas, reduciendo el riesgo de inyección SQL)
- Mantenibilidad (el modelo de datos vive junto a la lógica del dominio, en el mismo lenguaje)
- Migraciones (muchos ORMs versionan y aplican cambios de esquema de forma controlada)
- Menos boilerplate (evita mapear manualmente resultados de consultas a objetos)

Desventajas

- Curva de aprendizaje (dominar bien un ORM lleva tiempo)
- Overhead de rendimiento (es menos eficiente que SQL)
- Consultas complejas (consultas avanzadas son difíciles de expresar)
- Abstracción con fugas (hay que entender el SQL que se genera)
- Problema N+1 (un patron de acceso mal escrito puede disparar consultas innecesarias)
- Menos control fino (índices, hints y optimizaciones específicas del motor son más difíciles de aplicar)

Active Record	Data Mapper
El objeto sabe cómo persistir a sí mismo	La persistencia vive separada del objeto
usuario.save()	session.add(usuario)
usuario.delete()	session.delete(usuario)
usuario.find(id)	session.query(Usuario)
Simple y directo, pero mezcla lógica de negocio con persistencia	Objetos de dominio más puros; requiere una capa extra (mapper / session).

ORM populares

- Python -> SQLAlchemy - Django ORM

- Java / Kotlin -> Hibernate - JPA
- .NET -> Entity Framework Core
- JavaScript / TS -> Prisma - TypeORM - Sequelize
- Ruby -> Active Record (Rails)
- PHP -> Eloquent (Laravel) - Doctrine

El problema N+1 se soluciona utilizando eager loading para hacer el JOIN.

Migración: versionar el esquema

- Cambiar el modelo (un campo nuevo, por ejemplo).
- Generar migración (la herramienta detecta el cambio y crea script).
- Aplicar en cada entorno (el script se corre en desarrollo, staging y producción).

¿Cuándo usar ORM?

Buen encaje	Usar con cautela
CRUD estándar de una aplicación	Reportes analíticos y agregaciones masivas
Prototipado y desarrollo rápido	Consultas de altísimo rendimiento crítico
Equipos con distintos niveles SQL	Migraciones de datos muy específicas
Portabilidad entre motores de BDD	BDD legacy con esquemas complejos
Apps con lógica de dominio rica	Operaciones batch sobre millones de filas

En resumen

- Un ORM traduce entre el mundo de objetos y el mundo relacional.
- Ahorra tiempo y reduce errores, pero no reemplaza entender SQL.
- Active Record para simplicidad - Data Mapper para separar mejor el dominio de la persistencia.
- Vigilar el problema N+1 y usar eager loading cuando se accede a relaciones en un bucle.
- Las migraciones son tan parte del ORM como las consultas: versionan el esquema junto al código.

ASP NET CORE - CONTROLLERS / DI

Program.cs: el punto de entrada (es el único lugar donde se decide que implementación usar)

- `builder.Services` = es el contenedor de Inyección de Dependencias: ahí se registra todo lo que la app va a necesitar.
- `builder.Build()` = arma la aplicación con esos servicios ya configurados.
- Después de `Build()` = se define el pipeline: la secuencia de middlewares que procesa cada request.

Inyección de Dependencias

Sin DI	Con DI
El controller crea sus propias dependencias: <code>new ApplicationDbContext(...)</code>	El controller declara que necesita en su constructor
Queda acoplado a una implementación concreta	El contenedor de DI (registrado en Program.cs) se encarga de crearlo e inyectarlo
Si la forma de crear la dependencia cambia, hay que tocar cada controller	El controller solo conoce la abstracción (IMapper), no como se construye
Difícil de testear (no se puede reemplazar el DbContext por uno falso)	Fácil de testear: se puede implementar una versión de prueba

La solución es recibir la dependencia desde afuera

- **Inversión de Control (IoC)** = quien decide cuando se crea la dependencia - ya no la clase.
- **Inyección de Dependencias (DI)** = como se la entrego a la clase que la necesita.

El árbol arma el contenedor de la siguiente manera

ProductoController	pide <code>IProductoService</code>
ProductoService	pide <code>IProductoRepository</code>

ProductoRepository	pide ApplicationDbContext
AppDbContext	se construye con la receta de Program.cs

El contenedor arma todo de atrás para adelante y entre el ProductoController ya listo. Lo arma solo; siguiendo lo que pide cada constructor.

Cuando algo pide un IProductoServe, .NET entrega una instancia de ProductoService, y si esa clase necesita un ApplicationDbContext, se lo resuelve solo.

Registrando servicios en el contenedor

AddControllers	Habilita el uso de controllers y su infraestructura
AddDbContext<T>	Registra el DbContext como scoped, lito para inyectarse
AddAutoMapper	Registra IMapper con los perfiles del mapeo del proyecto
AddSwaggerGen	Registra la documentación interactiva (Swagger / OpenAPI)

Ciclos de vida de un servicio

Transient	Scoped	Singleton
Instancia nueva cada vez que se pide	Una instancia por request HTTP	Una sola instancia para toda la app
Un servicio stateless (como un generador de tokens)	AppDbContext - todo lo que pasa en un mismo request comparte la misma conexión	Configuración, cache en memoria, logger

AppDbContext ¿porque no Singleton?

“Si registro ApplicationDbContext como Singleton, ahorro crear instancias...?”

- **Singleton:** DbContext no es thread-safe. Dos request concurrentes comparten la misma instancia y se pisan los datos (transacciones mezcladas, entidades trackeadas incorrectamente, etc).

- **Scoped:** Una instancia por request. Todo lo que pasa dentro de un mismo request HTTP comparte la misma conexión y el mismo contexto de tracking. Cada request nuevo arranca limpio, sin pisar a los demás.

Se registra una interfaz y no la clase para poder cambiar la implementación real sin tocar el código que la consume.

El pipeline de middlewares - cada request atraviesa esta secuencia, **en este exacto orden**, antes de llegar (o no) al controller.

- **UseSwagger / UI** = expone la documentación interactiva.
- **UseHttpsRedirection** = redirige requests HTTP a HTTPS.
- **UseAuthorization** = evalúa políticas de autorización.
- **MapControllers** = conecta las rutas de los controllers con el pipeline. Recién en este paso puede entrar un request a una acción.

Anatomía de un Controller

- **[ApiController]** -activa comportamientos automáticos: validación de ModelState, inferencia de binding (ruta / query / body) y respuestas 400 automáticas ante datos inválidos-
- **[Route("api/generos")]** -define el prefijo de ruta para todas las acciones de la clase-
- **ControllerBase** -base para APIs (sin soporte de vistas). Da acceso a Ok(), NotFound(), BadRequest(), etc-

Ruteo por atributos

- **[Route("api/generos")]** = ruta fija a nivel de clase: prefijo para todas las acciones.
- **[HttpPut("{id:int}")]** = parametro de ruta con restricción de tipo.
- **[HttpPost("varios")]** = sub-ruta fija combinada con el prefijo de la clase.
- **[Route("api/peliculas/{peliculaid:int}/comentarios")]** = recurso anidado: los comentarios cuelgan de una película.

Model binding: ¿de donde viene cada dato?

[ApiController] infiere el origen de cada parámetro sin que lo indiquemos explícitamente.

Ruta	[HttpGet("{id:int}")] Get(int id)	id viene de {id} en la URL: /api/actores/5
Query string	[HttpGet("nombre")] Get(string nombre)	nombre no está en la ruta -> se busca en la URL: ?nombre=Tom
Cuerpo (Body)	[HttpPost] Post(GeneroCreacionDTO dto)	Un tipo complejo (DTO) se toma del body en JSON
Ruta + Body	Post(int peliculald, ComentarioCreacionDTO dto)	peliculald viene de la ruta de la clase; dto, del body

ActionResult<T> en ASP.NET Core es un tipo de retorno para controladores de API web que permite devolver un objeto de un tipo específico o cualquier resultado HTTP válido.

Cuatro controllers, un mismo patrón: constructor con DbContext + IMapper, acciones async, ActionResult<T>.

- GenerosController = CRUD clásico completo: alta simple, alta en lote, reemplazo y dos formas de borrar (con y sin tracking).
- PeliculasController = Alta con relaciones (géneros y actores) y dos formas de consultas: con Include o con proyección.
- ActoresController = Filtros por query string (nombre exacto vs "contiene") y una proyección directa a DTO.
- ComentariosController = Recurso anidado bajo Pelicula: la ruta expresa la relación entre entidades.